Part I, Week 01, Day 07

Matrix Calculus Notation Rigor & Backpropagation Foundations

Abstract

এখন পর্যন্ত calculus আমরা mainly use করেছি slope আর curvature বোঝার জন্য।
কিন্তু neural network train করতে গেলে millions of parameters থাকে, তাই এক এক করে derivative বের করা সম্ভব না।
আমাদের একসাথে **matrix** আর **tensor** differentiate করতে হয়।

এখানেই আসে **Matrix Calculus Notation Rigor** — মানে derivative-এর notation আর shape ঠিকভাবে বুঝতে হবে।
এটা clear হলে আমরা **Computational Graph** আর **Backpropagation** বুঝতে পারব, যা modern AI-এর main engine।

1) The Great Divide: Matrix Calculus Notation

যদি y scalar আর x scalar হয়, তাহলে:

dy/dx

একটা scalar-ই হয়।
কিন্তু যদি vector কে vector দিয়ে differentiate করি, বা scalar কে weight matrix দিয়ে করি, তখন derivative-এর shape কী হবে — এটাই confusion।

Matrix calculus-এ **2**টা **convention** আছে।
এই convention না বুঝলে **dimension mismatch error** হয়।

1.1 Numerator Layout (Jacobian Convention)

এখানে derivative-এর shape নির্ভর করে **output**-এর উপর।

Scalar w.r.t Vector

যদি y scalar হয়, আর x size n column vector হয়, তাহলে:

dy/dx

হবে:

1 x n

অর্থাৎ **row vector**।

Vector w.r.t Vector

যদি y size m আর x size n, তাহলে:

dy/dx

হবে:

m x n

এটাই **Jacobian Matrix**।

1.2 Denominator Layout (Gradient Convention)

এখানে derivative-এর shape নির্ভর করে **input**-এর উপর।

Scalar w.r.t Vector

যদি y scalar আর x size n column vector হয়, তাহলে:

dy/dx

হবে:

n x 1

অর্থাৎ **column vector**।
এটাই standard gradient vector।

Vector w.r.t Vector

যদি `y` size `m` আর `x` size `n`, তাহলে derivative হবে:

n x m

এটা Jacobian-এর transpose।

1.3 The Machine Learning Compromise

Modern deep learning frameworks practical way-তে দুই convention-ই use করে।

1) Loss gradient এর জন্য Denominator Layout

কারণ আমরা চাই:

dL/dW

এর shape যেন `W`-এর same হয়।
তাহলে update সহজ হয়:

W_new = W_old - eta * dL/dW

2) Intermediate layers এর জন্য Numerator Layout

কারণ vector-to-vector derivatives এ Jacobian use করলে chain rule clean ভাবে matrix multiplication দিয়ে লেখা যায়।

2) The Generalized Chain Rule for Vectors

ধরি neural network-এ:

h = f(x)
y = g(h)

এখানে `x -> h -> y`

তাহলে chain rule অনুযায়ী:

dy/dx = (dy/dh) * (dh/dx)

যদি:

x : n-dimensional
h : k-dimensional
y : m-dimensional

তাহলে:

dy/dh : m x k
dh/dx : k x n

তাই:

dy/dx : m x n

এটা perfectly input x থেকে output y mapping represent করে।

3) Computational Graphs: The Canvas of Backpropagation

একটা neural network আসলে অনেক বড় nested mathematical expression।
এটাকে আমরা **Computational Graph** দিয়ে visualize করি।
এটা basically একটা **Directed Acyclic Graph (DAG)**।

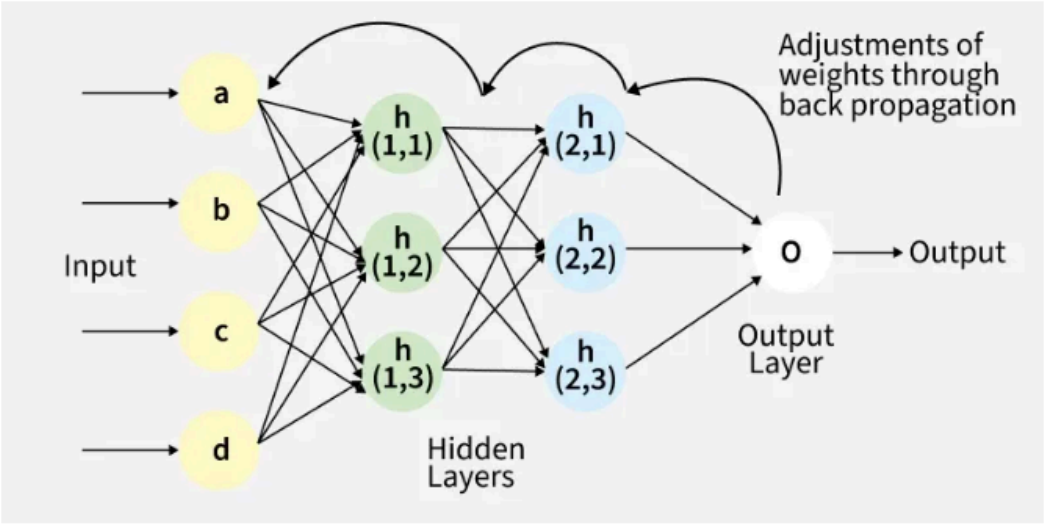
Nodes

Nodes represent করে:

- variables (input, weight)
- operations (addition, multiplication, ReLU, Sigmoid)

Edges

Edges দেখায় tensor/data এক node থেকে next node-এ কীভাবে flow করছে।



3.1 The Forward Pass

Forward pass-এ data **left to right** যায়।
প্রতিটি node নিজের operation compute করে, আর intermediate values store করে।

এই stored values cache করতে হয়, কারণ backward pass-এ local derivative বের করতে এগুলো লাগবে।

3.2 The Backward Pass

Backward pass-এ gradients **right to left** যায়, অর্থাৎ loss থেকে input-এর দিকে ফিরে আসে।

প্রতিটি node-এ main rule হলো:

Downstream Gradient = Upstream Gradient * Local Gradient

Terms

- **Upstream Gradient** = next layer থেকে আসা loss gradient
- **Local Gradient** = এই node-এর নিজের derivative
- **Downstream Gradient** = এদের product, যা previous node-এ পাঠানো হয়

এটাই chain rule-এর practical use।

4) The Mechanics of Backpropagation: A Micro-Example

ধরি একটা multiplication node আছে:

z = x * y

আর final loss হলো L।

Step 1: Forward Pass

ধরি:

x = 3
y = 4

তাহলে:

z = 12

Step 2: Upstream Gradient আসে

Backward pass-এ ধরি node-টা পেল:

dL/dz = 5

এটাই upstream gradient।

Step 3: Local Gradients বের করি

যেহেতু:

z = x * y

তাই:

dz/dx = y = 4
dz/dy = x = 3

Step 4: Downstream Gradients বের করি

Chain rule use করে:

dL/dx = (dL/dz) * (dz/dx) = 5 * 4 = 20
dL/dy = (dL/dz) * (dz/dy) = 5 * 3 = 15

অর্থাৎ node backward pass-এ:

- x branch-এ 20 পাঠায়
- y branch-এ 15 পাঠায়

এভাবেই ছোট ছোট node-level calculation মিলিয়ে full backpropagation হয়।

Extra intuition

Addition node

যদি node হয়:

z = x + y

তাহলে:

dz/dx = 1
dz/dy = 1

তাই upstream gradient দুই input-এ same ভাবে distribute হয়।

Multiplication node

যদি node হয়:

z = x * y

তাহলে:

- x-এর gradient scale হয় y দিয়ে
- y-এর gradient scale হয় x দিয়ে

এই জন্য multiplication node-কে gradient switcher বলা হয়।